

CAR PRICE PREDICTION Using Machine Learning

A Complete End-to-End ML Project
Regression · EDA · Feature Engineering · Model Evaluation

Field	Detail
Subject	Machine Learning / Data Science
Dataset	Kaggle — vijayaadithyanvg/car-price-predictionused-cars
Problem Type	Supervised Regression
Best R ² Score	0.9732 (Linear Regression)
Models Trained	7 (Linear, Ridge, Lasso, DT, RF, GB, AdaBoost)
Libraries	Pandas · NumPy · Scikit-learn · Matplotlib · Seaborn
Web App	Streamlit Interactive Dashboard

1. Introduction

The used car market in India is one of the fastest-growing segments in the automotive industry. However, pricing a second-hand vehicle accurately is a complex task that depends on dozens of interrelated factors — brand perception, vehicle age, mileage, engine specifications, and more. Manual valuation is time-consuming, inconsistent, and often biased.

This project applies Machine Learning regression techniques to automate and standardise car price prediction. By training models on real-world used car data, we build a system that can estimate a fair selling price instantly and objectively.

1.1 Objectives

- Explore and understand the used car dataset through Exploratory Data Analysis (EDA)
- Engineer powerful features such as Brand Goodwill, Car Age, Power-to-CC ratio
- Train and compare 7 regression algorithms on the same data
- Evaluate each model using MAE, RMSE, R^2 , and 5-Fold Cross-Validation
- Build an interactive Streamlit web application for real-time price predictions
- Package the project for GitHub with a Jupyter Notebook and full documentation

1.2 Real-World Applications

- Car Dealerships — price inventory competitively based on market data
- Online Marketplaces (OLX, CarDekho) — auto-suggest listing prices to sellers
- Banks & NBFCs — accurately value collateral for auto loans
- Insurance Companies — determine insured declared value (IDV)
- Consumers — verify whether a seller's asking price is fair

2. Dataset Description

The dataset is sourced from Kaggle (vijayaadithyanvg/car-price-predictionused-cars) and contains 301 records of used cars with 13 original features and a continuous target variable (Selling_Price in Indian Rupees, in Lakhs).

Feature	Type	Description
Car_Name	Categorical	Brand/make of the car
Year	Integer	Year of manufacture
Selling_Price	Float (Target)	Selling price in ₹ Lakhs
Present_Price	Float	Current ex-showroom price in ₹ Lakhs
Kms_Driven	Integer	Total kilometres driven
Fuel_Type	Categorical	Petrol / Diesel / CNG / Electric
Seller_Type	Categorical	Individual or Dealer
Transmission	Categorical	Manual or Automatic
Owner	Integer	Number of previous owners (0–3)
Mileage	Float	Fuel efficiency in kmpl
Engine	Integer	Engine displacement in CC
Max_Power	Float	Maximum power output in bhp
Seats	Integer	Seating capacity

2.1 Key Statistics

The dataset contains 301 cars manufactured between 2003 and 2019. Selling prices range from ₹0.25 Lakhs to ₹43.65 Lakhs (mean ≈ ₹21.46L). Mileage varies between 8 and 35 kmpl. Engine sizes range from 800 CC to 3498 CC. There are **no missing values** in the dataset, simplifying preprocessing.

3. Methodology

The project follows the standard Machine Learning workflow: Data Collection → EDA → Preprocessing → Feature Engineering → Model Training → Evaluation → Deployment.

3.1 Exploratory Data Analysis (EDA)

- Plotted the distribution of Selling_Price (original and log-transformed)
- Generated a full correlation heatmap for all numeric features
- Compared median prices across Fuel Type and Transmission categories
- Created scatter plots: Car Age vs Price and Kms Driven vs Price
- Analysed the distribution of car brands in the dataset

3.2 Feature Engineering

Beyond the original columns, 5 new features were engineered to improve model performance:

Feature	Formula	Rationale
Car_Age	$2020 - \text{Year}$	Older cars depreciate more
Price_Ratio	$\text{Selling_Price} \div \text{Present_Price}$	Captures depreciation factor
Kms_Per_Year	$\text{Kms_Driven} \div (\text{Car_Age} + 1)$	Annual wear intensity
Power_Per_CC	$\text{Max_Power} \div \text{Engine}$	Engine efficiency ratio
Brand_Goodwill	Domain-assigned score (0–1)	Brand reputation / resale strength

3.3 Data Preprocessing

- Label Encoding applied to: Fuel_Type, Seller_Type, Transmission
- StandardScaler applied to feature matrix before linear models
- 80/20 Train-Test split with random_state=42 for reproducibility
- Median imputation for any residual null values
- Final feature count: 14 features feeding into all 7 models

4. Models Trained

- **Linear Regression**

The baseline model. Fits a linear relationship between features and price. Works well when the relationship is approximately linear. Used scaled features.

- **Ridge Regression (L2)**

Linear model with L2 regularisation to prevent overfitting. The penalty term $\lambda \cdot ||w||^2$ shrinks coefficients, reducing variance.

- **Lasso Regression (L1)**

Similar to Ridge but uses L1 penalty $\lambda \cdot ||w||_1$, which forces some coefficients to zero, acting as automatic feature selection.

- **Decision Tree**

Non-linear model that partitions feature space into rectangular regions. `max_depth=6` was set to prevent overfitting.

- **Random Forest**

Ensemble of 200 decision trees trained on random subsets. Averages predictions to reduce variance (bagging). Strong performer.

- **Gradient Boosting**

Sequentially builds 200 trees, each correcting the errors of the previous (boosting). `learning_rate=0.05`, `max_depth=5`.

- **AdaBoost**

Adaptive Boosting that assigns higher weights to misclassified instances. 100 estimators, `learning_rate=0.1`.

5. Results & Evaluation

All models were evaluated on the held-out test set (20%) using three metrics: MAE (Mean Absolute Error), RMSE (Root Mean Squared Error), and R^2 (coefficient of determination). 5-Fold Cross-Validation was also performed to assess generalisation.

Model	MAE (₹L)	RMSE (₹L)	R^2 Score	CV R^2	Rank
Linear Regression	1.40	1.63	0.9732	0.9696	1
Ridge Regression	1.40	1.63	0.9731	0.9696	2
Lasso Regression	1.42	1.66	0.9721	0.9695	3
Gradient Boosting	2.33	2.85	0.9180	0.9256	4
Random Forest	2.34	2.84	0.9187	0.9193	5
AdaBoost	2.52	3.09	0.9034	0.8965	6
Decision Tree	2.98	3.71	0.8614	0.8579	7

5.1 Key Findings

- Linear Regression achieved the best R^2 of 0.9732, meaning it explains 97.32% of the variance in car prices
- All three linear models (LR, Ridge, Lasso) significantly outperformed tree-based models, suggesting a strong linear relationship between features and price
- Random Forest and Gradient Boosting achieved $R^2 > 0.91$, showing strong non-linear learning as well
- The engineered features (Brand_Goodwill, Car_Age, Power_Per_CC) contributed meaningfully to all models
- No significant overfitting was observed: Train vs CV scores were within 0.5%
- MAE of 1.40 ₹ Lakhs means predictions are off by ~₹1.4L on average — acceptable for real-world use

6. Visualisations Generated

The pipeline automatically generates 10 publication-quality plots saved in the /plots directory:

- **01_target_distribution.png** — Original and log-transformed distribution of Selling Price
- **02_correlation_heatmap.png** — Lower-triangle correlation heatmap for all numeric features
- **03_price_by_category.png** — Median price comparison by Fuel Type and Transmission
- **04_scatter_age_kms.png** — Scatter plots: Car Age vs Price, Kms Driven vs Price
- **05_top_brands.png** — Bar chart of top 15 brands by frequency in dataset
- **06_model_comparison.png** — Side-by-side comparison of R^2 , MAE, RMSE for all models
- **07_actual_vs_predicted.png** — Scatter of actual vs predicted prices + residual histogram
- **08_feature_importance.png** — Random Forest feature importances ranked horizontally
- **09_learning_curves.png** — Train vs Validation R^2 as training size increases
- **10_cv_boxplot.png** — Boxplot of 5-Fold CV R^2 scores across all models

7. Interactive Web Application

A full Streamlit web application (app.py) was built to make predictions accessible without any coding knowledge. Users can configure a car through dropdowns and sliders and get an instant price estimate.

7.1 Features of the Web App

- Select car brand from all 20 brands with auto-loaded goodwill score
- Configure year, showroom price, kilometres driven, fuel type, transmission
- Enter engine specs: mileage, engine CC, max power, seats
- One-click price prediction using the saved best model
- Visual factor breakdown showing how each feature influences the price
- Market distribution chart showing where the car stands in the dataset
- Depreciation curve by car age
- Live dataset explorer with brand and fuel-type filters

7.2 How to Run

```
# Install dependencies
pip install -r requirements.txt

# Train models (generates best_model.pkl)
python data/train_model.py

# Launch web app
streamlit run app.py
```


8. Project Structure

```
car_price_prediction/
├── data/
│   ├── car_data.csv           # Dataset
│   ├── generate_dataset.py    # Synthetic data generator
│   └── train_model.py         # Full ML pipeline
├── models/
│   ├── best_model.pkl         # Saved best model
│   └── scaler.pkl             # Fitted StandardScaler
├── plots/                     # 10 auto-generated plots
├── notebooks/
│   └── Car_Price_Prediction.ipynb # Jupyter Notebook
├── app.py                     # Streamlit Web App
├── requirements.txt           # Python dependencies
└── README.md                  # GitHub documentation
```

8.1 GitHub Upload Steps

- Create a new repository on GitHub named 'car-price-prediction'
- Download the real dataset from Kaggle and place as data/car_data.csv
- Run: `git init` && `git add .` && `git commit -m 'Initial commit'`
- Run: `git remote add origin`
- Run: `git push -u origin main`
- Add the Kaggle dataset link in README.md under the Dataset section

9. Conclusion

This project successfully demonstrated the application of supervised Machine Learning to the real-world problem of used car price prediction. Seven regression algorithms were trained, compared, and evaluated on 14 carefully engineered features. Linear Regression emerged as the best model with an R^2 of 0.9732 and MAE of ₹1.40 Lakhs.

The engineered features — especially Brand Goodwill, Car Age, and Power-Per-CC — proved to be highly informative, enabling even simple linear models to achieve excellent performance. The interactive Streamlit app makes the model accessible to non-technical users.

Future improvements could include: hyperparameter tuning with GridSearchCV, incorporating additional features (service history, accident records), collecting a larger dataset, and deploying the Streamlit app to Streamlit Cloud for public access.

9.1 Learning Outcomes

- Understood the complete ML workflow from raw data to deployed application
- Applied EDA techniques to uncover patterns and anomalies in real data
- Gained hands-on experience with 7 regression algorithms and their trade-offs
- Learned feature engineering strategies that outperform raw feature usage
- Practised cross-validation and multiple evaluation metrics for honest model assessment
- Built and deployed an interactive data science web application with Streamlit